

建议学生亲自安装所选用的 RDBMS，观察数据库安装过程，记录和理解安装配置参数，熟悉 RDBMS 的客户端图形管理工具。

熟悉选定的数据库实验环境（如 RDBMS 或专门的数据库实验实训平台）。

第3章实验 SQL 查询与操纵

SQL 查询与操纵实验包含 6 个实验项目（见表 4），其中 5 个必修实验项目，1 个选修实验项目。实验 3.1～实验 3.5 为设计性实验，实验 3.6 为验证性实验。

表 4 “SQL 查询与操纵”实验项目一览表

序号	实验项目名称	开设类别	难易程度	建议实验学时	对应教材章节
实验 3.1	数据库定义	必修	适中	2	第 3 章 3.2 节
实验 3.2	数据基本查询	必修	适中	2	第 3 章 3.3 节
实验 3.3	数据高级查询	必修	较难	2	第 3 章 3.3 节
实验 3.4	数据更新	必修	适中	2	第 3 章 3.4 节
实验 3.5	视图	必修	适中	2	第 3 章 3.6 节
实验 3.6	索引	选修	适中	2	第 3 章 3.2.3 小节

实验 3.1 数据库定义

1. 实验内容和要求

理解并掌握数据定义语言 DDL 语法和各种参数的具体含义及使用方法，能够熟练地使用 DDL 语句来创建、修改和删除数据库模式和基本表，掌握 DDL 语句常见语法错误及其调试方法。

实验重点：学会创建数据库和基本表。

实验难点：创建基本表时，为不同的列选择合适的数据类型；正确创建表级和列级完整性约束，如列值是否允许为空、主码和外码等。

实验过程中要注意数据完整性约束既可以在创建基本表时定义，也可以先创建表然后再定义完整性约束；由于完整性约束的限制，被引用的表要先创建。

2. 实验报告示例

实验报告				
题目：数据库定义	姓名		日期	
本实验建立零件供应销售数据库模式（见图 1），包含零件表（Part）、供应商表（Supplier）、零件供应联系表（PartSupp）、顾客表（Customer）、国家表（Nation）、地区表（Region）、订单表（Orders）和订单明细表（Lineitem）8 个基本表。				

续表

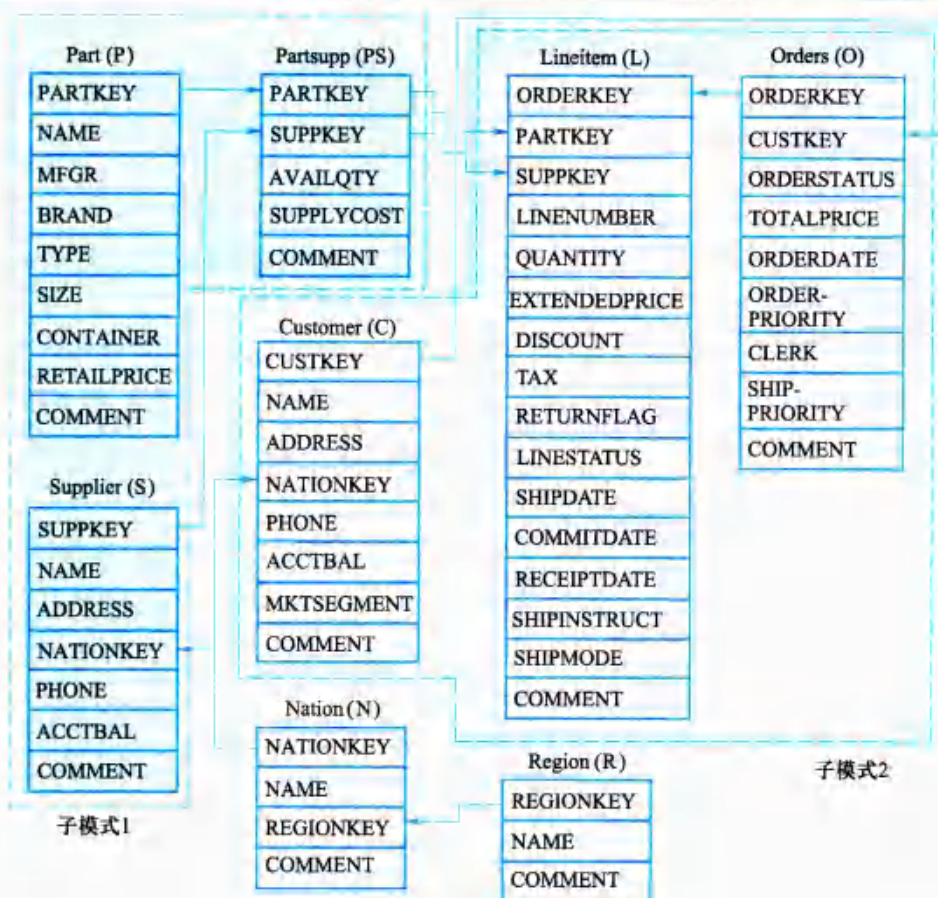


图1 零件供应销售数据库模式图

零件供应销售数据库模式又可以分为两个子模式。

子模式1：零件供应商子模式，包括 Part、Supplier 和 PartSupp 三个表，类似学生、课程和选课数据库模式，该子模式还可以增加 Nation 和 Region 两个表。该模式中 Part 和 Supplier 之间是多对多类型的联系。

子模式2：顾客订单子模式，包括 Customer、Orders 和 Lineitem 三个表，该子模式也可以增加 Nation 和 Region 两个表。该模式中各实体之间主要是一对多类型的联系。

后续各个实验项目的示例基于零件供应销售数据库模式，或者是基于上述两个子模式。

(1) 定义数据库

采用中文字符集创建名为零件供应销售 (part supply sales, PSS) 的数据库。

```
CREATE DATABASE PSS ENCODING = 'GBK';
```


续表

(2) 定义模式

在零件供应销售数据库中创建名为 Sales 的模式。

```
CREATE SCHEMA Sales;
```

(3) 定义基本表

在零件供应销售数据库的 Sales 模式中创建 8 个表。

```
/*设置当前会话的搜索路径为 Sales 模式、Public 模式，表会自动创建在 Sales 模式下*/
```

```
SET SEARCH_PATH TO Sales, Public;
```

```
CREATE TABLE Region ( /*地区表*/
```

```
    Regionkey INTEGER PRIMARY KEY, /*地区编号*/
```

```
    Name CHAR(25), /*地区名称*/
```

```
    Comment VARCHAR(152) /*备注*/);
```

```
CREATE TABLE Nation ( /*国家表*/
```

```
    Nationkey INTEGER PRIMARY KEY, /*国家编号*/
```

```
    Name CHAR(25), /*国家名称*/
```

```
    Regionkey INTEGER REFERENCES Region(regionkey), /*所属地区编号*/
```

```
    Comment VARCHAR(152) /*备注*/);
```

```
CREATE TABLE Supplier ( /*供应商表*/
```

```
    Suppkey INTEGER PRIMARY KEY, /*供应商编号*/
```

```
    Name CHAR(25), /*供应商名称*/
```

```
    Address VARCHAR(40), /*供应商地址*/
```

```
    Nationkey INTEGER REFERENCES Nation(nationkey), /*国家编号*/
```

```
    Phone CHAR(15), /*供应商电话*/
```

```
    Acctbal REAL, /*供应商账户余额 account balance*/
```

```
    Comment VARCHAR(101) /*备注*/);
```

```
CREATE TABLE Part ( /*零件表*/
```

```
    Partkey INTEGER PRIMARY KEY, /*零件编号*/
```

```
    Name VARCHAR(55), /*零件名称*/
```

```
    Mfgr CHAR(25), /*制造商*/
```

```
    Brand CHAR(10), /*品牌*/
```

```
    Type VARCHAR(25), /*类型*/
```

```
    Size INTEGER, /*尺寸*/
```

```
    Container CHAR(10), /*包装*/
```

```
    Retailprice REAL, /*零售价格*/
```

```
    Comment VARCHAR(23) /*备注*/);
```

```
CREATE TABLE PartSupp ( /*零件供应联系表*/
```

续表

```

Partkey INTEGER REFERENCES Part(Partkey),      /*零件编号*/
Supkey INTEGER REFERENCES Supplier(Supkey),     /*供应商编号*/
Availqty INTEGER,                             /*可用数量*/
Supplycost REAL,                             /*供应价格*/
Comment VARCHAR(199),                         /*备注*/
PRIMARY KEY(Partkey,Supkey)                   /*定义主码,表级约束*/);

CREATE TABLE Customer ( /*顾客表*/
    Custkey INTEGER PRIMARY KEY,                /*顾客编号*/
    Name VARCHAR(25),                          /*姓名*/
    Address VARCHAR(40),                       /*地址*/
    Nationkey INTEGER REFERENCES Nation(Nationkey), /*国籍编号*/
    Phone CHAR(15),                            /*电话*/
    Acctbal REAL,                              /*账户余额*/
    Mktsegment CHAR(10),                      /*市场分区*/
    Comment VARCHAR(117)                      /*备注*/);

CREATE TABLE Orders ( /*订单表*/
    Orderkey INTEGER PRIMARY KEY,              /*订单编号*/
    Custkey INTEGER REFERENCES Customer(Custkey), /*顾客编号*/
    Orderstatus CHAR(1),                      /*订单状态*/
    Totalprice REAL,                         /*订单总金额*/
    Orderdate DATE,                         /*订单日期*/
    Orderpriority CHAR(15),                 /*订单优先级*/
    Clerk CHAR(15),                        /*记账员*/
    Shippriority INTEGER,                  /*运输优先级*/
    Comment VARCHAR(79)                   /*备注*/);

/*其中 Totalprice = SUM(Lineitem.Extendedprice × (1-Lineitem.Discount) × (1+ Lineitem.Tax)) */

CREATE TABLE Lineitem ( /*订单明细表*/
    Orderkey INTEGER REFERENCES Orders(Orderkey), /*订单编号*/
    Partkey INTEGER REFERENCES Part(Partkey),    /*零件编号*/
    Supkey INTEGER REFERENCES Supplier(Supkey),  /*供应商编号*/
    Linenumber INTEGER,                       /*订单明细编号*/
    Quantity REAL,                          /*数量*/
    Extendedprice REAL,                    /*订单明细价格*/
    Discount REAL,                        /*折扣[0.00, 1.00]*/
    Tax REAL,                            /*税率[0.00, 0.08]*/
    Returnflag CHAR(1),                   /*退货标记*/
    Linestatus CHAR(1),                   /*订单明细状态*/

```

续表

```

Shipdate DATE,           /*装运日期*/
Commitdate DATE,         /*委托日期*/
Receiptdate DATE,        /*签收日期*/
Shipinstruct CHAR(25),   /*装运说明, 如 deliver in person*/
Shipmode CHAR(10),       /*装运方式, 如空运、陆运、铁运和海运等*/
Comment VARCHAR(44),     /*备注*/
PRIMARY KEY(Orderkey,Linenumber),
FOREIGN KEY (Partkey,Suppkey) REFERENCES PartSupp(Partkey,Suppkey) );
/*其中订单明细价格 Extendedprice = Quantity×Part.Retailprice*/

```

实验总结:

- ① 初学者可以先定义零件供应商子模式, 包括 Part、Supplier 和 PartSupp 三个表, 类似学生、课程和选课数据库模式。
- ② 初学者可以先不定义实体完整性和参照完整性, 待讲完有关概念后再在实验 3 中练习。

3. 思考题

- ① SQL 语法规则, 双引号括定的符号串为对象名称, 单引号括定的符号串为常量字符串。请思考什么情况下用双引号界定对象名, 并进行实验验证。
- ② 数据库对象的完整引用是“服务器名.数据库名.模式名.对象名”, 但通常可以省略服务器名和数据库名, 甚至省略模式名, 直接用对象名访问对象即可。请设计相应的实验, 验证基本表及其列的访问方法。

实验 3.2 数据基本查询

1. 实验内容和要求

掌握 SQL 程序设计基本规范, 熟练运用 SQL 实现数据基本查询, 包括单表查询、分组统计查询和连接查询; 针对零件供应销售数据库, 设计各种单表查询 SQL 语句、分组统计查询语句; 设计单表针对自身的连接查询, 设计多表的连接查询; 理解和掌握 SQL 查询语句中各个子句的特点和作用, 按照 SQL 程序设计规范, 写出具体的 SQL 查询语句并调试通过。

实验重点: 分组统计查询、单表自身连接查询、多表连接查询。

实验难点: 区分元组过滤条件和分组过滤条件; 确定连接属性, 正确设计连接条件。

实验过程中要注意遵循 SQL 程序设计规范, 包含 SQL 关键字大写, 表名、属性名、存储过程名等标识符大小写混合, SQL 程序书写缩进排列等编程规范。具体内容请参见参考文献[6]。

2. 实验报告示例

实验报告			
题目：数据基本查询	姓名	日期	
(1) 单表查询（实现投影操作） 查询供应商的名称、地址和联系电话。 <pre>SELECT Name, Address, Phone FROM Supplier;</pre>			
(2) 单表查询（实现选择操作） 查询最近一周内提交的总价大于 1000 元的订单编号、顾客编号等订单的所有信息。 <pre>SELECT * FROM Sales.Orders /*模式名.表名，Sales 模式中的 Orders 表*/ WHERE CURRENT_DATE - Orderdate < 7 and Totalprice > 1000; /*选择条件*/</pre>			
(3) 不带分组过滤条件的分组统计查询 统计每个顾客的订购金额。 <pre>SELECT C.Custkey, SUM(O.Totalprice) /*对每个组，运用聚集函数 SUM*/ FROM Customer C, Orders O WHERE C.Custkey = O.Custkey GROUP BY C.Custkey; /*按照 C.Custkey 分组*/</pre>			
(4) 带分组过滤条件的分组统计查询 查询订单平均金额超过 1 000 元的顾客编号及其姓名。 <pre>SELECT C.Custkey, MAX(C.Name) /*分组属性和聚集函数才能出现在 SELECT 子句中*/ FROM Customer C, Orders O WHERE C.Custkey = O.Custkey GROUP BY C.Custkey /*按照 C.Custkey 分组*/ HAVING AVG(O.Totalprice) > 1000; /*按照条件对组进行过滤，只输出满足条件的组*/</pre>			
(5) 单表自身连接查询 查询与“金仓集团”在同一个国家的供应商编号、名称和地址信息。 <pre>SELECT F.Suppkey, F.Name, F.Address FROM Supplier F, Supplier S /* Supplier 表的自身连接*/ WHERE F.Nationkey = S.Nationkey AND S.Name = '金仓集团';</pre>			
(6) 两表连接查询（普通连接） 查询供应价格大于零售价格的零件名、制造商名、零售价格和供应价格。			

续表

```
SELECT P.Name, P.Mfg, P.Retailprice, PS.Supplycost
FROM Part P, PartSupp PS          /* 两表连接, 给表起个别名, 简化表达 */
WHERE P.Retailprice > PS.Supplycost; /* 限定条件 */
```

/*说明: 上述连接语句是从 2 个表的笛卡儿积中选出满足限定条件的元组, 得到的结果可能不是同一个商品的有关值, 所以应该改为下面的自然连接*/

(7) 两表连接查询 (自然连接)

查询供应价格大于零售价格的零件名、制造商名、零售价格和供应价格。

```
SELECT P.Name, P.Mfg, P.Retailprice, PS.Supplycost
FROM Part P, PartSupp PS          /* 两表连接, 给表起个别名, 以简化表达 */
WHERE P.Partkey = PS.Partkey      /* 连接条件 */
AND P.Retailprice > PS.Supplycost; /* 限定条件 */
```

(8) 三表连接查询

查询顾客“苏举库”订购的订单编号、总价及其订购的零件编号、数量和明细价格。

```
SELECT O.Orderkey, O.Totalprice, L.Partkey, L.Quantity, L.Extendedprice
FROM Customer C, Orders O, Lineitem L
WHERE C.Custkey = O.Custkey AND O.Orderkey = L.Orderkey AND C.Name='苏举库';
```

实验总结:

- ① 正确理解数据库模式结构, 才能正确设计数据库查询。
- ② 连接查询是数据库 SQL 查询中最重要的查询, 要特别注意连接查询的设计, 不同的查询表达, 其查询执行的性能会有很大差别。

3. 思考题

- ① 不在 GROUP BY 子句出现的属性, 是否可以出现在 SELECT 子句中? 请举例并上机验证。
- ② 请举例说明分组统计查询中的 WHERE 和 HAVING 有何区别。
- ③ 连接查询速度是影响关系数据库性能的关键因素。请讨论如何提高连接查询速度, 并通过实验进行验证。

实验 3.3 数据高级查询

1. 实验内容和要求

掌握 SQL 嵌套查询和集合查询等各种高级查询的设计方法; 正确分析用户查询要求, 设计各种嵌套查询和集合查询。

实验重点: 掌握嵌套查询。

实验难点: 掌握相关子查询、多层 EXISTS 嵌套查询。

2. 实验报告示例

实验报告

题目：数据高级查询

姓名

日期

(1) IN 嵌套查询

查询订购了“海大”制造的“船舶模拟驾驶舱”的顾客。

```
SELECT Custkey, Name
FROM Customer
WHERE Custkey IN (SELECT O.Custkey
FROM Orders O, Lineitem L, PartSupp PS, Part P /*四表连接*/
WHERE O.Orderkey = L.Orderkey AND
      L.Partkey = PS.Partkey AND
      L.Suppkey = PS.Suppkey AND
      PS.Partkey = P.Partkey AND
      P.Mfgr = '海大' AND P.Name = '船舶模拟驾驶舱');
/*试比较上述查询与下列查询有何区别。下列查询中 Lineitem 表直接与 Part 表连接*/
/*请见实验总结*/
```

```
SELECT Custkey, Name
FROM Customer
WHERE Custkey IN (SELECT O.Custkey
FROM Orders O, Lineitem L, Part P /*三表连接*/
WHERE O.Orderkey = L.Orderkey AND
      L.Partkey = P.Partkey AND
      P.Mfgr = '海大' AND P.Name = '船舶模拟驾驶舱');
```

(2) 单层 EXISTS 嵌套查询

查询没有购买过“海大”制造的“船舶模拟驾驶舱”的顾客。

```
SELECT Custkey, Name
FROM Customer C
WHERE NOT EXISTS (SELECT *
/*购买“海大”制造的“船舶模拟驾驶舱”的顾客*/
FROM Orders O, Lineitem L, PartSupp PS, Part P
WHERE C.Custkey = O.Custkey AND /*此条件为相关子查询条件*/
      O.Orderkey = L.Orderkey AND
      L.Partkey = PS.Partkey AND
      L.Suppkey = PS.Suppkey AND
      PS.Partkey = P.Partkey AND
      P.Mfgr = '海大' AND P.Name = '船舶模拟驾驶舱');
```


续表

(3) 双层 EXISTS 嵌套查询

查询至少购买过顾客“张三”购买过的全部零件的顾客姓名。

```
SELECT CA.Name /*查找 CA 客户, 这位客户没有张三购买过而自己买过的零件*/
FROM Customer CA
WHERE NOT EXISTS
    (SELECT * /*张三购买过而 CA 客户没有买过的零件*/
    FROM Customer CB, Orders OB, Lineitem LB
    WHERE CB.Custkey = OB.Custkey AND
          OB.Orderkey = LB.Orderkey AND
          CB.Name = '张三' AND
          NOT EXISTS (SELECT * /*CA 客户与 CB 客户都购买过的零件*/
                     FROM Orders OC, Lineitem LC
                     WHERE CA.Custkey = OC.Custkey AND
                           OC.Orderkey = LC.Orderkey AND
                           LB.Suppkey = LC.Suppkey AND
                           LB.Partkey = LC.Partkey ));
```

(4) FROM 子句中的嵌套查询, 即基于派生表的查询

查询在订单平均金额超过 1 万元的顾客中, 中国籍顾客的信息。

```
SELECT C.*
FROM Customer C, (SELECT Custkey /*子查询生成的临时派生表为 B(custkey)表*/
FROM Orders
GROUP BY Custkey
HAVING AVG(Totalprice) > 10000) AS B(Custkey), Nation N
WHERE C.Custkey = B.Custkey AND /*B 表成为主查询的查询对象*/
      C.Nationkey = N.Nationkey AND N.Name = '中国';
```

(5) 集合查询 (交)

查询顾客“张三”和“李四”都订购过的全部零件的信息。

```
SELECT P.*
FROM Customer C, Orders O, Lineitem L, PartSupp PS, Part P
WHERE C.Custkey = O.Custkey AND O.Orderkey = L.Orderkey AND
      L.Suppkey = PS.Suppkey AND L.Partkey = PS.Partkey AND
      PS.Partkey = P.Partkey AND C.Name = '张三';

INTERSECTION
SELECT P.*
FROM Customer C, Orders O, Lineitem L, PartSupp PS, Part P
WHERE C.Custkey = O.Custkey AND O.Orderkey = L.Orderkey AND
```

续表

```
L.Suppkey = PS.Suppkey AND L.Partkey = PS.Partkey AND  
PS.Partkey = P.Partkey AND C.Name = '李四';
```

(6) 集合查询(并)

查询顾客“张三”和“李四”订购的全部零件的信息。

```
SELECT P.*  
FROM Customer C,Orders O, Lineitem L, PartSupp PS, Part P  
WHERE C.Custkey = O.Custkey AND O.Orderkey = L.Orderkey AND  
L.Suppkey = PS.Suppkey AND L.Partkey = PS.Partkey AND  
PS.Partkey = P.Partkey AND C.Name = '张三';  
UNION  
SELECT P.*  
FROM Customer C,Orders O, Lineitem L, PartSupp PS, Part P  
WHERE C.Custkey = O.Custkey AND O.Orderkey = L.Orderkey AND  
L.Suppkey = PS.Suppkey AND L.Partkey = PS.Partkey AND  
PS.Partkey = P.Partkey AND C.Name = '李四';
```

(7) 集合查询(差)

顾客“张三”订购过而“李四”没订购过的零件的信息。

```
SELECT P.*  
FROM Customer C,Orders O, Lineitem L, PartSupp PS, Part P  
WHERE C.Custkey = O.Custkey AND O.Orderkey = L.Orderkey AND  
L.Suppkey = PS.Suppkey AND L.Partkey = PS.Partkey AND  
PS.Partkey = P.Partkey AND C.Name = '张三';  
EXCEPT  
SELECT P.*  
FROM Customer C,Orders O, Lineitem L, PartSupp PS, Part P  
WHERE C.Custkey = O.Custkey AND O.Orderkey = L.Orderkey AND  
L.Suppkey = PS.Suppkey AND L.Partkey = PS.Partkey AND  
PS.Partkey = P.Partkey AND C.Name = '李四';
```

实验总结:

① 通过分析图 1 中零件供应销售数据库模式可知, Lineitem 表是通过 PartSupp 表和 Part 表联系的, 所以“IN 嵌套查询”中的第一个查询是正常的查询表达方式。而由于 Partkey 是 Part 表的主码, 第二个查询也能得出相同的结果。因此, 生成 Lineitem 表记录时利用 PartSupp 表保证供应商和零件的一致性, 而查询 Lineitem 表时可以直接和 Part 表相连接。同样, 也可以直接和 Suppliers 表相连接。

② 相关子查询和不相关子查询, 将在“数据库查询性能调优”实验中进一步讲解和实验。

3. 思考题

- ① 请分析什么类型的查询可以用连接查询实现, 什么类型的查询只能用嵌套查询实现。
- ② 请分析不相关子查询和相关子查询的区别。

实验 3.4 数据更新

1. 实验内容和要求

熟悉数据库的数据更新操作, 能够使用 SQL 语句对数据库进行数据的插入、修改、删除操作; 针对零件供应销售数据库, 设计单元组插入、批量数据插入、修改数据和删除数据等 SQL 语句; 理解和掌握 INSERT、UPDATE 和 DELETE 语法结构的各个组成部分, 结合嵌套 SQL 子查询, 分别设计几种不同形式的插入、修改和删除数据的语句, 并调试成功。

实验重点: 设计插入、修改和删除数据的 SQL 语句。

实验难点: 设计与嵌套 SQL 子查询相结合的插入、修改和删除数据的 SQL 语句; 利用一个表的数据插入、修改和删除另外一个表的数据。

2. 实验报告示例

实验报告			
题目: 数据更新	姓名	日期	
(1) INSERT 基本语句 (插入全部列的数据) 插入一条顾客记录, 要求每列都给一个合理的值。 <pre>INSERT INTO Customer VALUES(30, '张三', '北京市', 40, '010-51001199', 0.00, 'Northeast', 'VIP Customer');</pre>			
(2) INSERT 基本语句 (插入部分列的数据) 插入一条订单记录, 给出必要的几个字段值。 <pre>INSERT INTO Lineitem(Orderkey, Linenumber, Partkey, Suppkey, Quantity, Shipdate) VALUES (862, ROUND(RANDOM()*100,0), 479, 1, 10, '2012-3-6'); /*RANDOM() 函数为随机小数生成函数, ROUND() 函数为四舍五入函数*/</pre>			
(3) 批量数据 INSERT 语句 ① 创建一个新的顾客表, 把所有中国籍顾客插入新的顾客表。 <pre>CREATE TABLE NewCustomer AS SELECT * FROM Customer WITH NO DATA; /*WITH NO DATA 子句使得 SELECT 查询只生成一个结果模式 NewCustomer, 不查询出实际数据*/ INSERT INTO NewCustomer /*把 SELECT 语句查询结果批量插入 NewCustomer 表*/ SELECT C.* FROM Customer C, Nation N /*给 Customer 和 Nation 取别名 C 和 N */ WHERE C.Nationkey = N.Nationkey AND N.Name = '中国';</pre>			

续表

- ② 创建一个顾客购物统计表, 记录每个顾客及其购物总数和总价等信息。

```
CREATE TABLE ShoppingStat
( Custkey INTEGER,
  Quantity REAL,
  Totalprice REAL);
INSERT INTO ShoppingStat
SELECT C.Custkey, Sum(L.Quantity), Sum (O.Totalprice)    /*对分组后的数据求总和*/
FROM   Customer C, Orders O, Lineitem L
WHERE  C.Custkey = O.Custkey AND O.Orderkey = L.Orderkey
GROUP BY C.Custkey;
```

- ③ 倍增零件表的数据, 多次重复执行, 直到总记录数达到 50 万条为止。

```
INSERT INTO Part
SELECT Partkey + (SELECT COUNT(*) FROM Part),
       Name, Mfgr, Brand, Type, Size, Container, Retailprice, Comment
FROM Part;
```

说明:

① 该方法是迅速生成大量数据的一个有效方法, 但其缺点是要求主码为数值类型并且除了主码外其他属性上没有 Unique 约束, 因此该方法生成的数据具有很大的重复性。

② 生成 50 万条记录所要执行上述 INSERT 语句的次数, 需根据 Part 表的初始记录数通过人工计算。

- ③ 执行上述 INSERT 语句的速度会一次比一次慢, 因此需要耐心等待。

(4) UPDATE 语句 (修改部分记录的部分列值)

“金仓集团”供应的所有零件的供应成本价下降 10%。

```
UPDATE PartSupp
SET Supplycost = Supplycost * 0.9
WHERE Suppkey = (SELECT Suppkey    /*找出要修改的那些记录*/
FROM Supplier
WHERE Name = '金仓集团');
```

(5) UPDATE 语句 (利用一个表中的数据修改另外一个表中的数据)

利用 Part 表中的零售价格来修改 Lineitem 中的 Extendedprice, 其中 $\text{Extendedprice} = \text{Part.Retailprice} * \text{Quantity}$ 。

```
UPDATE Lineitem L
SET L.Extendedprice = P.Retailprice * L.Quantity
FROM Part P
WHERE L.Partkey = P.Partkey
```

续表

/*Lineitem 表可以直接与 Part 表相连接, 无须通过 PartSupp 连接*/

(6) DELETE 基本语句 (删除给定条件的所有记录)

删除顾客张三的所有订单记录。

```
DELETE FROM Lineitem      /*先删除张三的订单明细记录*/
WHERE Orderkey IN (SELECT Orderkey
FROM Orders O, Customer C
WHERE O.Custkey = C.Custkey AND C.Name = '张三');
DELETE FROM Orders        /*再删除张三的订单记录*/
WHERE Custkey = (SELECT Custkey
FROM Customer
WHERE Name = '张三');
```

实验总结:

① 只有正确设计和执行数据更新语句, 并确保正确录入数据和更新数据, 才能保证查询出来的数据正确。

② 当更新数据失败时, 一个主要原因是更新数据时违反了完整性约束, 如更新主码导致主码重复, 更新外码值导致它不能在参照表中找到相应的主码值等。

3. 思考题

① 请分析数据库模式更新和数据更新 SQL 语句的异同。

② 请分析数据库系统除了 INSERT、UPDATE 和 DELETE 等基本的数据更新语句之外, 还有哪些可以用来更新数据库基本表数据的 SQL 语句。

实验 3.5 视图

1. 实验内容和要求

熟悉 SQL 有关视图的操作, 能够熟练使用 SQL 语句来创建需要的视图, 定义数据库外模式, 并能使用所创建的视图实现数据管理; 针对给定的数据库模式和相应的应用需求, 创建视图、创建带 WITH CHECK OPTION 的视图, 并验证视图 WITH CHECK OPTION 选项的有效性; 理解和掌握视图消解执行原理, 掌握可更新视图和不可更新视图的区别。

实验重点: 创建视图。

实验难点: 掌握可更新的视图和不可更新的视图之区别, 以及 WITH CHECK OPTION 选项有效性的验证。

2. 实验报告示例

实验报告

题目：视图

姓名

日期

(1) 创建视图（省略视图列名）

创建一个“海大汽配”供应商供应的零件视图 V_DLMU_PartSupp1，要求列出供应零件的编号、零件名称、可用数量、零售价格、供应价格和备注等信息。

```
CREATE VIEW V_DLMU_PARTSUPP1 AS      /*由 SELECT 子句目标列组成视图属性*/
SELECT P.Partkey, P.Name, PS.Availqty, P.Retailprice, PS.Supplycost, P.Comment
FROM Part P, PartSupp PS, Supplier S
WHERE P.Partkey=PS.Partkey AND S.Supkey=PS.Supkey AND S.Name='海大汽配';
```

(2) 创建视图（不能省略列名的情况）

创建一个视图 V_CustAvgOrder，按顾客统计平均每个订单的购买金额和零件数量，要求输出顾客编号、姓名、平均购买金额和平均购买零件数量。

```
CREATE VIEW V_CustAvgOrder(Custkey,Cname,Avgprice,Avgquantity) AS
SELECT C.Custkey, MAX(C.Name), AVG(O.Totalprice), AVG(L.Quantity)
/*因为 C.name 没有出现在 Group By 子句中，如需要出现在 SELECT 子句中，则必须使用 MAX
函数，否则就会出现语法错误*/
FROM Customer C, Orders O, Lineitem L
WHERE C.Custkey = O.Custkey AND L.Orderkey = O.Orderkey
GROUP BY C.Custkey;
```

(3) 创建视图(WITH CHECK OPTION)

使用 WITH CHECK OPTION，创建一个“海大汽配”供应商供应的零件视图 V_DLMU_PartSupp2，要求列出供应零件的编号、可用数量和供应价格等信息，然后通过该视图分别增加、删除和修改一条“海大汽配”零件供应记录，验证 WITH CHECK OPTION 是否起作用。

```
CREATE VIEW V_DLMU_PartSupp2
AS
SELECT Partkey, Supkey, Availqty, Supplycost
FROM PartSupp
WHERE Supkey = (SELECT Supkey
                FROM Supplier
                WHERE name = '海大汽配')
WITH CHECK OPTION;

INSERT INTO V_DLMU_PartSupp2
VALUES(58889, 5048, 704, 77760);
```


续表

```
UPDATE V_DLMU_PartSupp2
SET Supplycost = 12
WHERE Suppkey = 58889;

DELETE FROM V_DLMU_PartSupp2
WHERE Suppkey = 58889;
```

(4) 可更新的视图（行列子集视图）

创建一个“海大汽配”供应商供应的零件视图 V_DLMU_PartSupp3，要求列出供应零件的编号、可用数量和供应价格等信息，然后通过该视图分别增加、删除和修改一条“海大汽配”零件供应记录，验证该视图是否可以更新，并比较第 3 题与本题结果有何异同。

```
CREATE VIEW V_DLMU_PartSupp3
AS
SELECT Partkey, Suppkey, Availqty, Supplycost
FROM PartSupp
WHERE Suppkey = (SELECT Suppkey
                  FROM Supplier
                  WHERE Name = '海大汽配');

INSERT INTO V_DLMU_PartSupp3
VALUES(58889,5048,704,77760);

UPDATE V_DLMU_PartSupp3
SET Supplycost = 12
WHERE Suppkey = 58889;

DELETE FROM V_DLMU_PartSupp3
WHERE Suppkey = 58889;
```

(5) 不可更新的视图

第(2)题中创建的视图是可更新的吗？通过 SQL 更新语句加以验证，并说明原因。

```
INSERT INTO V_CustAvgOrder
VALUES(100000,NULL,20,2000);
```

(6) 删除视图（RESTRICT/CASCADE）

创建顾客订购零件明细视图 V_CustOrd，要求列出顾客编号、姓名、购买零件数、金额；

续表

在该视图的基础上，再创建第 2 题的视图 V_CustAvgOrder，然后使用 RESTRICT 选项删除视图 V_CustOrd，观察现象并解释原因。利用 CASCADE 选项删除视图 V_CustOrd，观察现象并检查 V_CustAvgOrder 是否存在。解释原因。

```
CREATE VIEW V_CustOrd(Custkey,Cname,Qty,Extprice)
AS
SELECT C.Custkey, C.Name, L.Quantity, L.Extendedprice
FROM Customer C,Orders O,Lineitem L
WHERE C.Custkey = O.Custkey AND O.Orderkey = L.Orderkey;
CREATE VIEW V_CustAvgOrder(Custkey,Cname, Avgqty, Avgprice)
AS
SELECT Custkey,MAX(Cname),AVG(Qty),AVG(Extprice)
FROM V_CustOrd      /*在视图 V_CustOrd 上再创建视图*/
GROUP BY Custkey;

DROP VIEW V_CustOrd RESTRICT;

DROP VIEW V_CustOrd CASCADE;
```

实验总结：

视图的重要作用包括实现数据库外模式、简化查询、清晰表达查询、实现数据安全保护等，设计合适的视图也是数据库设计的一项重要内容；要掌握可更新视图和不可更新视图的区别与联系；不同的 DBMS 对视图的定义和实现各不相同，使用具体 DBMS 时，还需要参考相应 DBMS 的 SQL 手册。

5. 思考题

- ① 请分析视图和基本表在使用方面有哪些异同，并设计相应的示例加以验证。
- ② 请具体分析修改基本表的结构对相应的视图会产生何种影响。

实验 3.6 索引

1. 实验内容和要求

掌握索引的设计原则和技巧，能够创建合适的索引以提高数据库查询、统计分析效率；针对给定的数据库模式和具体应用需求，创建唯一索引、函数索引、复合索引等，修改索引，删除索引；设计相应的 SQL 查询，验证索引的有效性。学习利用 EXPLAIN 命令分析 SQL 查询是否使用了所创建的索引，并能够分析其原因，执行 SQL 查询并估算索引提高查询效率的百分比。要求实验数据集达到 10 万条记录以上的数据量，以便验证索引效果。

实验重点：创建索引。

实验难点：设计 SQL 查询以验证索引的有效性。

2. 实验报告示例

实验报告			
题目：索引	姓名	日期	
(1) 创建唯一索引 在零件表的零件名称字段上创建唯一索引。 <pre>CREATE UNIQUE INDEX Idx_part_name ON Part (Name);</pre> (2) 创建函数索引（对某个属性的函数创建索引，称为函数索引） 在零件表的零件名称字段上创建一个零件名称长度的函数索引。 <pre>CREATE INDEX Idx_part_name_fun ON Part (LENGTH(Name));</pre> (3) 创建复合索引（对两个及两个以上的属性创建索引，称为复合索引） 在零件表的制造商和品牌两个字段上创建一个复合索引。 <pre>CREATE UNIQUE INDEX Idx_part_mfgr_brand ON Part (Mfgr, Brand);</pre> (4) *创建聚簇索引 在零件表的制造商字段上创建一个聚簇索引。 <pre>CREATE UNIQUE INDEX Idx_part_mfgr ON Part (Mfgr); CLUSTER Idx_part_mfgr ON Part;</pre> /*有关聚簇索引的概念在《概论》第7章7.5.2小节“选择关系模式存取方法”中介绍*/ (5) 创建哈希索引 在零件表的名称字段上创建一个哈希索引。 <pre>CREATE INDEX Idx_part_name_hash ON Part USING HASH (Name);</pre> (6) 修改索引名称 修改零件表的名称字段上的索引名。 <pre>ALTER INDEX Idx_part_name_hash RENAME TO Idx_part_name_hash_new;</pre> (7) 分析某个 SQL 查询语句执行时是否使用了索引 <pre>EXPLAIN SELECT * FROM Part WHERE Name = '零件';</pre>			

续表

/*EXPLAIN 是查看 SQL 查询性能的最基本的方法, 可提供查询优化器 (query optimizer) 关于处理语句的执行计划的详细信息*/

(8) *验证索引效率

创建一个函数 TestIndex, 自动计算 SQL 查询执行的时间。

```
CREATE FUNCTION TestIndex(p_partname CHAR(55)) RETURN INTEGER
AS /*自定义函数 TestIndex(): 输入参数为零件名称, 返回 SQL 查询的执行时间*/
DECLARE
    begintime    TIMESTAMP;
    endtime      TIMESTAMP;
    durationtime INTEGER;
BEGIN
    SELECT CLOCK_TIMESTAMP() INTO begintime;    /*记录查询执行的开始时间*/
    PERFORM SELECT * FROM Part WHERE Name = p_partname;
                                                /*执行 SQL 查询, 不保存查询结果*/
    SELECT CLOCK_TIMESTAMP() INTO endtime;      /*记录查询执行的结束时间*/
    SELECT DATEDIFF('ms', begintime, endtime) INTO durationtime;
    RETURN durationtime;                      /*计算并返回查询执行时间, 时间单位为 ms*/
END;

/*查看当零件表 Part 数据规模比较小, 并且无索引时的执行时间*/
SELECT TestIndex('零件名称');
INSERT INTO Part /*不断倍增零件表的数据, 直到 50 万条记录*/
SELECT Partkey + (SELECT COUNT(*) FROM Part),
       Name, Mfgr, Brand, Type, Size, Container, Retailprice, Comment
FROM Part;

/*查看当零件表 Part 数据规模比较大, 并且无索引时的执行时间*/
SELECT TestIndex('零件名称');

CREATE INDEX part_name ON Part(Name);    /*在零件表的零件名称字段上创建索引*/

/*查看零件表 Part 数据规模比较大, 但是有索引时的执行时间*/
SELECT TestIndex('零件名称');
```

比较上述各次执行时间, 可计算出索引提高查询效率的百分比。

续表

实验总结:

索引属于数据库物理设计范畴。设计合适的索引可以显著地提高数据查询的效率;但是,一种类型的索引并不能提高所有查询的效率,因此需要搞清楚不同类型的索引各有什么特点,以及对什么样的查询起作用。

1. 思考题

- ① 在一个表的多个字段上创建的复合索引,与在相应的每个字段上创建的多个简单索引有何异同?请设计相应的示例加以验证。
- ② B+树索引、哈希索引、聚簇索引各自有什么特点,各自对什么样的查询起作用?
- ③ 请参阅《概论》第9章中关于B+树索引、哈希索引的讲解。

第4章实验 安全性控制

安全性控制实验包含两个实验项目(见表5),其中1个必修实验项目,1个选修实验项目。自主存取控制实验为设计性实验,审计实验为验证性实验。本实验指导目前没有设置强制存取控制和数据加密等方面的实验项目。

表5 “安全性控制”实验项目一览表

序号	实验项目名称	开设类别	难易程度	建议实验学时	对应教材章节
实验4.1	自主存取控制实验	必修	适中	2	第4章4.2节
*实验4.2	审计实验	选修	适中	2	第4章4.4节

实验4.1 自主存取控制实验

1. 实验内容和要求

掌握自主存取控制权限的定义和维护方法。定义用户、角色,分配权限给用户、角色,回收权限;以相应的用户名登录数据库,验证权限分配是否正确;选择一个应用场景,使用自主存取控制机制设计权限分配。可以采用两种方案:

方案一为采用SYSTEM超级用户登录数据库,完成所有权限分配工作,然后用相应用户名登录数据库以验证权限分配的正确性。

方案二为采用SYSTEM用户登录数据库,创建三个部门经理用户并分配相应的权限,然后分别用三个经理用户名登录数据库,创建相应部门的用户、角色并分配适当权限。

下面的实验报告示例采用了实验方案一。验证权限分配之前须备份数据库;针对不同用户所具有的权限,分别设计相应的SQL语句加以验证。

实验重点:定义角色,分配权限和回收权限。